LAPORAN PRAKTIKUM WORSHOP WEB DASAR
Javascript DOM

Nama :Muhammad Alfarisi

NIM :2025573010051

Kelas :TI /1C

PROGRAM STUDI TEKNIK INFORMATIKA JURUSAN INFORMASI DAN KOMPUTER POLITEKNIK NEGERI LHOKSEUMAWE

2026

DAFTAR ISI

BAB I
PENDAHULUAN

1. Latar Belakang

Lahirnya DOM berakar dari "Perang Browser" pada akhir 1990-an antara Netscape Navigator dan Microsoft Internet Explorer. Setiap browser menggunakan metode internalnya sendiri untuk berinteraksi dengan elemen web, sehingga pengembang kesulitan membuat situs web yang berfungsi baik di berbagai platform

2. Identifikasi Masalah

- Performa Lambat (Reder-Blocking & Reflow
- Celah Keamanan XSS (Cross-Site Scripting) berbasis DOM
- Masalah Skrip dan SEO (Crawl & Index)
- Kesalahan Navigasi dan Null Reference

3. Rumusan masalah

- Fokus pada Implementasi & Dasar
- Fokus pada Interaksi & Peristiwa (Events)
- Fokus pada Performa & Penerapan (Tingkat Lanjut)

BAB II
LANDASAN TEORI

1. Pengertian Javasript DOM

Javascript DOM (Document Object Model) adalah antamuka pemograman (API) yanng merepresentasikan halaman web dalam bentuk struktur pohon hierarkis. DOM memungkinkan bahasa pemograman seperti javaScript untuk mengakses, mengubah, menambah, atau menghapus elemen, konten, struktur, dan gaya dokumen HTML atau XML secara dinamis.

2. Struktur Javascript DOM

- Document Node: Titik akar (rott) dari seluruh pohon DOM.
- Element Node: Mewakili tag HTML seperti `<body>, <h1>, <p>, dan <div>.`
- Text Node: Mewakili isi text di dalam element HTML.
- Attribut Node: Mewakili atribut dari sebuah elemen (seperti `class, id, href`).

3. Fungsi Javascript DOM

Sebagai antarmuka pemograman yang menjebatani bahasa javascriot dengan documeny HTML atau XML.Hal ini memungkinkan Javascript untuk mengakses, mengubah, menambahkan, atau menghapsu element HTML, teks, maupun atribut untuk membangun halaman web yang interaktif.

BAB III
Paktikkum Dasar Tailwind.Css Dasar

1. Alat dan Bahan:

- Laptop
- Visual Studio Code
- Git
- GitHUb

2. Proses Pembuatan Laporan : Aplikasi Todo List Lengkap

Langkah 1 Buat folder Tugas pada modul -6

```
#Pastikan berada pada repository kamu
Mkdir modul-6
Cd modul-6
Mkdir tugas
```

Langkah 2 Buat file tugas index.html, tugas/style.css,dan tugas/script.js

```
Cd tugas
Touch index.html style.css script.js
```

Langkah 3 Isi code ini pada file index.html

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
    <meta  name="viewport"  content="width=device-width,  initial-
scale=1.0" />
```

```
  <title>Todo List - Modul 6</title>
  <link rel="stylesheet" href="styel.css" />
</head>
<body>
  <main class="page">
    <section class="hero">
      <div>
        <h1>Aplikasi Todo List</h1>
       <p>Tambah tugas dengan prioritas, tandai selesai, dan simpan
otomatis di localStorage.</p>
      </div>
      <div class="badge" id="todoCount">0 tugas tersisa</div>
    </section>

    <section class="task-form">
      <form id="addTaskForm">
        <div class="field-group">
          <label for="taskInput">Tugas</label>
           <input id="taskInput" type="text" placeholder="Masukkan
tugas..." maxlength="100" autocomplete="off" />
        </div>
        <div class="field-group">
          <label for="prioritySelect">Prioritas</label>
          <select id="prioritySelect">
            <option value="rendah">Rendah</option>
            <option value="sedang" selected>Sedang</option>
            <option value="tinggi">Tinggi</option>
          </select>
        </div>
             <button  type="submit"  class="primary-button">Tambah
Tugas</button>
      </form>
      <p class="help-text" id="formError" aria-live="polite"></p>
    </section>

    <section class="task-controls">
        <div  class="filter-group"  role="group"  aria-label="Filter
tugas">
         <button type="button" class="filter-button active" data-
filter="all">Semua</button>
              <button  type="button"  class="filter-button"  data-
filter="active">Aktif</button>
              <button  type="button"  class="filter-button"  data-
filter="completed">Selesai</button>
      </div>
               <button    type="button"    class="secondary-button"
id="clearCompletedButton">Hapus Semua Selesai</button>
    </section>
```

```
    <section class="task-list-section">
         <ul  class="task-list"  id="taskList"  aria-label="Daftar
tugas"></ul>
        <p  class="empty-state"  id="emptyState">Belum  ada  tugas.
Tambahkan tugas baru di atas.</p>
    </section>
  </main>

  <script src="script.js"></script>
</body>
</html>
```

Langkah 4 Isi code ini pada file style.css

```
:root {
  color-scheme: light;
  font-family: Inter, system-ui, sans-serif;
  font-size: 16px;
  color: #1f2937;
  background: #f0f4f8;
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  background: linear-gradient(180deg, #f0f4f8 0%, #e8ecf1 100%);
}

.page {
  max-width: 920px;
  margin: 0 auto;
  padding: 24px;
  display: grid;
  gap: 24px;
}

.hero {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  flex-wrap: wrap;
```

```
}

.hero h1 {
  margin: 0;
  font-size: clamp(2rem, 2.6vw, 2.8rem);
}

.hero p {
  margin: 0;
  color: #475569;
  line-height: 1.6;
  max-width: 640px;
}

.badge {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  padding: 0.85rem 1rem;
  border-radius: 999px;
  background: #e0f2fe;
  color: #0369a1;
  font-weight: 700;
}

.task-form,
.task-controls,
.task-list-section {
  background: rgba(255, 255, 255, 0.94);
  border: 1px solid rgba(148, 163, 184, 0.18);
  border-radius: 24px;
  padding: 22px;
  box-shadow: 0 18px 40px rgba(15, 23, 42, 0.06);
}

.field-group {
  display: grid;
  gap: 10px;
  margin-bottom: 18px;
}

.field-group label {
  font-weight: 700;
  color: #0f172a;
}

.field-group input,
.field-group select {
  width: 100%;
  border: 1px solid #cbd5e1;
```

```
  border-radius: 14px;
  padding: 12px 14px;
  font-size: 1rem;
  color: #0f172a;
  background: #ffffff;
}

.primary-button,
.secondary-button,
.icon-button {
  border: none;
  border-radius: 14px;
  cursor: pointer;
  font-weight: 700;
  transition: transform 0.15s ease, background 0.15s ease, box-shadow 0.15s ease;
}

.primary-button {
  background: #0f172a;
  color: white;
  padding: 0.95rem 1.2rem;
}

.primary-button:hover,
.secondary-button:hover,
.icon-button:hover {
  transform: translateY(-1px);
}

.secondary-button {
  background: #e2e8f0;
  color: #0f172a;
  padding: 0.95rem 1.2rem;
}

.task-controls {
  display: flex;
  flex-wrap: wrap;
  justify-content: space-between;
  align-items: center;
  gap: 14px;
}

.filter-group {
  display: inline-flex;
  gap: 10px;
  flex-wrap: wrap;
}

.filter-button {
```

```
  background: #f8fafc;
  color: #334155;
  padding: 0.85rem 1rem;
  border-radius: 14px;
}

.filter-button.active {
  background: #0f172a;
  color: white;
}

.task-list {
  margin: 0;
  padding: 0;
  list-style: none;
  display: grid;
  gap: 12px;
}

.task-item {
  display: grid;
  grid-template-columns: auto 1fr auto;
  align-items: center;
  gap: 14px;
  padding: 16px;
  border-radius: 20px;
  border: 1px solid #e2e8f0;
  background: #ffffff;
  cursor: grab;
}

.task-item.drag-over {
  border-color: #7dd3fc;
  box-shadow: 0 0 0 4px rgba(14, 165, 233, 0.12);
}

.task-item.completed {
  opacity: 0.8;
}

.task-checkbox {
  display: grid;
  place-items: center;
  width: 40px;
  height: 40px;
  border-radius: 14px;
  background: #f8fafc;
  position: relative;
}
```

```
.task-checkbox input {
  position: absolute;
  opacity: 0;
  pointer-events: none;
}

.checkmark {
  width: 18px;
  height: 18px;
  border: 2px solid #94a3b8;
  border-radius: 6px;
  display: inline-grid;
  place-items: center;
}

.task-checkbox input:checked + .checkmark {
  background: #0f172a;
  border-color: #0f172a;
}

.task-checkbox input:checked + .checkmark::after {
  content: '✓';
  color: white;
  font-size: 0.8rem;
}

.task-main {
  display: grid;
  gap: 10px;
}

.task-text {
  font-size: 1rem;
  line-height: 1.5;
  color: #0f172a;
  outline: none;
}

.task-text.editing {
  background: #f8fafc;
  border-radius: 12px;
  padding: 8px 10px;
}

.task-item.completed .task-text {
  text-decoration: line-through;
  color: #64748b;
}

.task-meta {
```

```
  display: flex;
  align-items: center;
  gap: 10px;
  flex-wrap: wrap;
}

.priority.badge {
  padding: 0.5rem 0.8rem;
  border-radius: 999px;
  color: white;
  font-size: 0.85rem;
}

.priority.rendah {
  background: #38bdf8;
}

.priority.sedang {
  background: #f97316;
}

.priority.tinggi {
  background: #ef4444;
}

.icon-button {
  min-width: 40px;
  min-height: 40px;
  display: inline-flex;
  align-items: center;
  justify-content: center;
  background: #e2e8f0;
  color: #0f172a;
}

.remove-button {
  background: #fee2e2;
  color: #b91c1c;
}

.empty-state,
.help-text {
  margin: 0;
  color: #475569;
}

.empty-state {
  display: none;
  padding: 18px;
  border-radius: 18px;
```

```
  background: #f8fafc;
  border: 1px dashed #cbd5e1;
  text-align: center;
}

.help-text {
  margin-top: 0;
  min-height: 1.2rem;
  color: #b91c1c;
}

@media (max-width: 720px) {
  .task-item {
    grid-template-columns: 1fr auto;
  }

  .task-controls {
    flex-direction: column;
    align-items: stretch;
  }
}
```

Langkah 5 Isi code ini pada file Script.js

```
const STORAGE_KEY = 'modul6-todo-list';

const form = document.getElementById('addTaskForm');
const taskInput = document.getElementById('taskInput');
const prioritySelect = document.getElementById('prioritySelect');
const formError = document.getElementById('formError');
const taskList = document.getElementById('taskList');
const emptyState = document.getElementById('emptyState');
const todoCount = document.getElementById('todoCount');
const filterButtons = document.querySelectorAll('.filter-button');
const                    clearCompletedButton                    =
document.getElementById('clearCompletedButton');

let tasks = [];
let currentFilter = 'all';
let draggedTaskId = null;

function loadTasks() {
  const saved = localStorage.getItem(STORAGE_KEY);
  tasks = saved ? JSON.parse(saved) : [];
}

function saveTasks() {
```

```
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

function getRemainingCount() {
  return tasks.filter(task => !task.completed).length;
}

function showError(message) {
  formError.textContent = message;
}

function clearError() {
  formError.textContent = '';
}

function formatPriority(priority) {
  return priority.charAt(0).toUpperCase() + priority.slice(1);
}

function createTaskHtml(task) {
  const doneClass = task.completed ? 'completed' : '';
  return `
      <li  class="task-item  ${doneClass}"  draggable="true"  data-
id="${task.id}">
      <label class="task-checkbox">
             <input  type="checkbox"  data-action="toggle"  data-
id="${task.id}" ${task.completed ? 'checked' : ''} />
        <span class="checkmark"></span>
      </label>
      <div class="task-main">
                 <span   class="task-text"   data-id="${task.id}"
tabindex="0">${escapeHtml(task.text)}</span>
        <div class="task-meta">
                                <span     class="priority     badge
${task.priority}">${formatPriority(task.priority)}</span>
            <button  type="button"  class="icon-button  edit-button"
data-action="edit"      data-id="${task.id}"      aria-label="Edit
tugas">Edit</button>
        </div>
      </div>
      <button type="button" class="icon-button remove-button" data-
action="delete"       data-id="${task.id}"       aria-label="Hapus
tugas">×</button>
    </li>
  `;
}

function escapeHtml(value) {
  return value
```

```
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#039;');
}

function renderTasks() {
  const filteredTasks = tasks.filter(task => {
    if (currentFilter === 'active') return !task.completed;
    if (currentFilter === 'completed') return task.completed;
    return true;
  });

  taskList.innerHTML = filteredTasks.map(createTaskHtml).join('');
  emptyState.style.display = filteredTasks.length === 0 ? 'block' :
'none';
  todoCount.textContent = `${getRemainingCount()} tugas tersisa`;
     clearCompletedButton.disabled    =    tasks.every(task    =>
!task.completed);
}

function getTaskIndex(id) {
  return tasks.findIndex(task => task.id === id);
}

function addTask(text, priority) {
  const trimmed = text.trim();
  if (!trimmed) return showError('Tugas tidak boleh kosong.');
   if  (trimmed.length  <  3)  return  showError('Tugas  minimal  3
karakter.');
   if (trimmed.length > 100) return showError('Tugas maksimal 100
karakter.');

  tasks.push({
    id: Date.now(),
    text: trimmed,
    priority,
    completed: false
  });

  saveTasks();
  renderTasks();
  taskInput.value = '';
  prioritySelect.value = 'sedang';
  clearError();
  taskInput.focus();
}
```

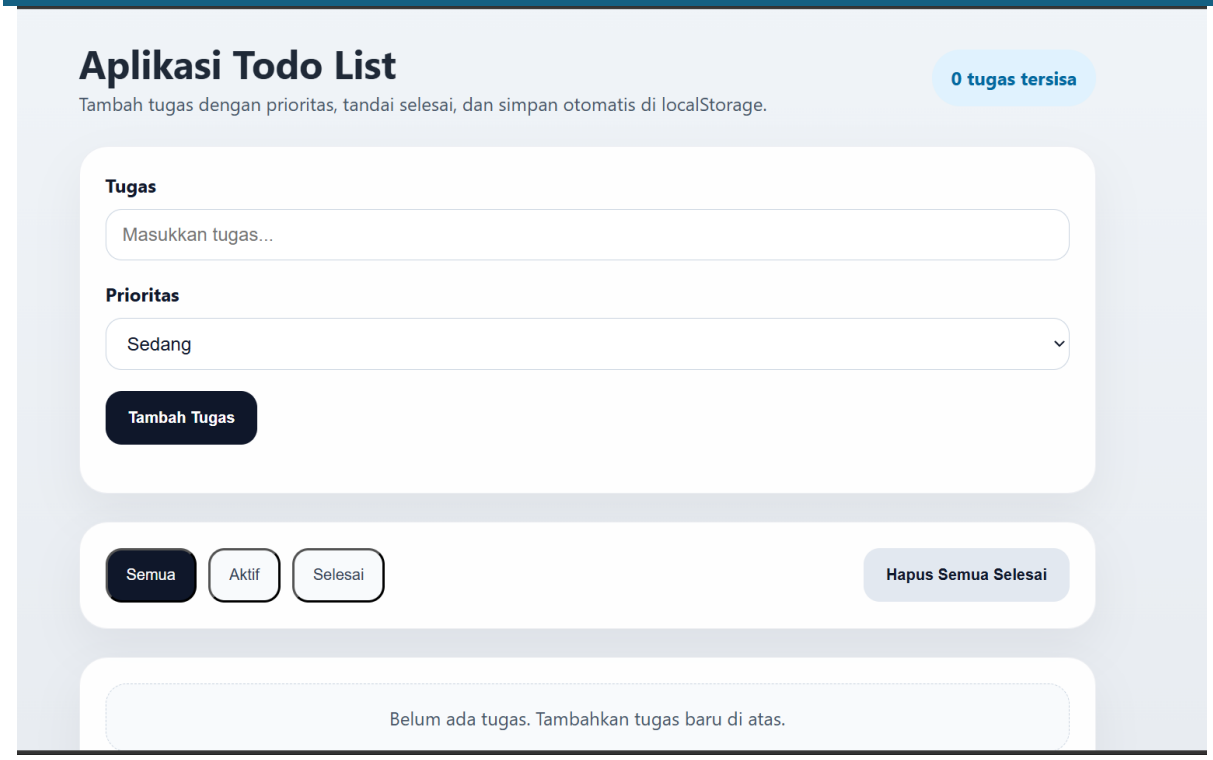
```
function toggleTaskCompletion(id) {
  const index = getTaskIndex(id);
  if (index === -1) return;
  tasks[index].completed = !tasks[index].completed;
  saveTasks();
  renderTasks();
}

function deleteTask(id) {
  tasks = tasks.filter(task => task.id !== id);
  saveTasks();
  renderTasks();
}

function clearCompletedTasks() {
  tasks = tasks.filter(task => !task.completed);
  saveTasks();
  renderTasks();
}

function updateTaskText(id, newText) {
  const trimmed = newText.trim();
  const index = getTaskIndex(id);
  if (index === -1) return;
  if (!trimmed) {
    showError('Tugas tidak boleh kosong.');
    renderTasks();
    return;
  }
  if (trimmed.length < 3) {
    showError('Tugas minimal 3 karakter.');
    renderTasks();
    return;
  }
  if (trimmed.length > 100) {
    showError('Tugas maksimal 100 karakter.');
    renderTasks();
    return;
  }

  tasks[index].text = trimmed;
  saveTasks();
  clearError();
  renderTasks();
}

function startTaskEdit(element, id) {
  const task = tasks.find(item => item.id === id);
  if (!task) return;
```

```
  element.contentEditable = 'true';
  element.classList.add('editing');
  element.focus();

  const range = document.createRange();
  range.selectNodeContents(element);
  const sel = window.getSelection();
  sel.removeAllRanges();
  sel.addRange(range);
}

function finishTaskEdit(element) {
  const id = Number(element.dataset.id);
  const newText = element.textContent || '';
  element.contentEditable = 'false';
  element.classList.remove('editing');
  updateTaskText(id, newText);
}

function setFilter(filter) {
  currentFilter = filter;
  filterButtons.forEach(button => {
      button.classList.toggle('active',  button.dataset.filter  ===
filter);
  });
  renderTasks();
}

function reorderTasks(draggedId, targetId) {
  const fromIndex = getTaskIndex(draggedId);
  const toIndex = getTaskIndex(targetId);
  if (fromIndex === -1 || toIndex === -1 || fromIndex === toIndex)
return;

  const [draggedTask] = tasks.splice(fromIndex, 1);
  tasks.splice(toIndex, 0, draggedTask);
  saveTasks();
  renderTasks();
}

form.addEventListener('submit', event => {
  event.preventDefault();
  addTask(taskInput.value, prioritySelect.value);
});

filterButtons.forEach(button => {
          button.addEventListener('click',          ()          =>
setFilter(button.dataset.filter));
```

```
});

clearCompletedButton.addEventListener('click',
clearCompletedTasks);

taskList.addEventListener('click', event => {
  const button = event.target.closest('button');
  if (button) {
    const action = button.dataset.action;
    const id = Number(button.dataset.id);
    if (action === 'delete') {
      deleteTask(id);
    }
    if (action === 'edit') {
       const textElement = taskList.querySelector(`.task-text[data-
id="${id}"]`);
      if (textElement) startTaskEdit(textElement, id);
    }
    return;
  }

  const checkbox = event.target.closest('input[type="checkbox"]');
  if (checkbox) {
    const id = Number(checkbox.dataset.id);
    toggleTaskCompletion(id);
  }
});

taskList.addEventListener('dblclick', event => {
  const textElement = event.target.closest('.task-text');
  if (!textElement) return;
  const id = Number(textElement.dataset.id);
  startTaskEdit(textElement, id);
});

taskList.addEventListener('keydown', event => {
  const textElement = event.target.closest('.task-text');
  if (!textElement) return;
  if (event.key === 'Enter') {
    event.preventDefault();
    textElement.blur();
  }
  if (event.key === 'Escape') {
    textElement.blur();
  }
});

taskList.addEventListener('blur', event => {
  const textElement = event.target.closest('.task-text');
```

```
  if (!textElement) return;
  finishTaskEdit(textElement);
}, true);

taskList.addEventListener('dragstart', event => {
  const item = event.target.closest('.task-item');
  if (!item) return;
  draggedTaskId = Number(item.dataset.id);
  event.dataTransfer.effectAllowed = 'move';
  event.dataTransfer.setData('text/plain', draggedTaskId);
});

taskList.addEventListener('dragover', event => {
  event.preventDefault();
  const item = event.target.closest('.task-item');
  if (!item) return;
  item.classList.add('drag-over');
});

taskList.addEventListener('dragleave', event => {
  const item = event.target.closest('.task-item');
  if (!item) return;
  item.classList.remove('drag-over');
});

taskList.addEventListener('drop', event => {
  event.preventDefault();
  const targetItem = event.target.closest('.task-item');
  if (!targetItem) return;
  targetItem.classList.remove('drag-over');

  const targetId = Number(targetItem.dataset.id);
                  const                  sourceId                  =
Number(event.dataTransfer.getData('text/plain'));
  if (sourceId && targetId && sourceId !== targetId) {
    reorderTasks(sourceId, targetId);
  }
});

window.addEventListener('DOMContentLoaded', () => {
  loadTasks();
  renderTasks();
});
```

Hasil Praktikum: membuat tugas Aplikasi todo list lengkap



BAB IV
PENUTUP

Kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa Javascript DOM memungkinkan pengembang untuk mengakses, memanipulasi, dan megubah konten, gaya (CSS), serta element halaman web secara dinamis menggunakan javaScript tanpa perlu memuat ulang halaman.